The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software
Unit Coverage: 1-5
Student Name: Vikalp Pratap Yadav
Registration Number: 24BCY10248
Slot: D11
Date of Submission: 31st March, 2026

1. Introduction

In the era of Digitalization where everything is computerized, the software makes an important part. Most of the software that powers the modern world—the servers that run websites, the tools that developers use every day, the operating systems inside supercomputers and smartphones—was not built by a single company and sold for profit. It was built openly, shared freely, and improved collectively by free minded people who were ready to contribute to make the digital world great without expecting anything in return. That is the essence of open source.

In this project we are going to focuses on reality instead of imaginations by exploring the hardships and what shortcoming are faced when an open source project are created back in the early days.

For this report, I have chosen to audit **Git**, the world's most ubiquitous version control system. I will conduct a structured audit of it, exploring not just what it does, but why it was born out of sheer necessity and frustration, who built it, what freedoms its General Public License (GPL) actually secures, the philosophical values it represents, and how it has fundamentally rewired the way human beings(developers and niche creaters) collaborate on digital projects.

Furthermore, apart from this written analysis, I have developed five shell scripts that demonstrate practical Linux administration skills. These scripts illustrate how open-source tools function at the command line inteface, proving that automation and transparency are not just theoretical concepts, but practical skills necessary for modern computing.

Every line of code written today sits on top of open-source foundations. Understanding the philosophy behind that foundation is an essential part of becoming a professional developer.

2. Choosing Your Software

For this audit, I have selected the following software from the approved list:

- **Software:** Git
- **Category:** Version Control
- **License:** GPL v2 (GNU General Public License version 2.0)
- **Why it's interesting:** It is the tool Linus Torvalds built over a single weekend when a proprietary software company revoked his community's free license, thereby, permanently changing how the world writes code and how private software companies/organization work.

3. Project Report Structure

PART A: Origin and Philosophy

A.1 The problem this software was created to solve

To understand reason behind the existence of Git, one must look at the chaotic landscape of software development before 2005, especially when during when programming has started. The Git which me see toady was not created to simply enter a market and compete with other product rather it was born out of an absolute, burning necessity to save the Linux kernel project from collapsing under its own weight.

The Pre-2002 Chaos: Tarballs and Patches

In the early days of the Linux kernel (1991-2002), there was no Automated Version Control System (AVCS). For those who don't know what is AVCS, it is an automated version control system which is a software tool designed to track and manage changes to source code or digital content over time, automatically handling revisions, merging contributions, and managing conflicts. ***Linus Torvalds***, the creator of Linux, received contributions from developers around the world in the form of unified diff files (patches) sent over email. He would manually review these text files, apply them to his local copy of the source code, and periodically release "tarballs" (compressed archives) of the entire kernel. As Linux exploded in popularity and the contributor base grew into the thousands, this manual process became unsustainable. Code was being lost, merge conflicts were a nightmare to resolve by hand, and the sheer volume of emails

was slowing down the pace of innovation.

Centralized version control systems existed at the time—namely CVS (Concurrent Versions System) and SVN (Subversion). However, Torvalds violently despised them. Centralized systems required a constant network connection to a single master server to commit code. If the server went down, nobody could work. Furthermore, the branching and merging which were the core activities of experimental software development—were incredibly slow and painful in SVN. Torvalds refused to use them, arguing they actively discouraged parallel development.

The BitKeeper Compromise

In 2002, the Linux kernel team struck a controversial deal. They began using BitKeeper, a proprietary, closed-source distributed version control system developed by Larry McVoy. McVoy offered the Linux community a free license to use BitKeeper, provided they did not attempt to reverse-engineer it or build a competing product. For three years, BitKeeper revolutionized Linux development. Because it was distributed, every developer had a full local copy of the repository, allowing them to branch, commit, and merge offline, only syncing with others when ready.

However, philosophical tension brewed. Richard Stallman and the Free Software Foundation heavily criticized Torvalds for relying on proprietary software to build the world's most important open-source project. They argued that the community was at the mercy of a corporate entity that could revoke their tools at any moment.

The 2005 Crisis and the Birth of Git

In April 2005, the critics were proven right. Andrew Tridgell (creator of Samba), frustrated by BitKeeper's limitations, began reverse-engineering BitKeeper's network protocols to create a free interoperability tool. Enraged by this violation of their agreement, Larry McVoy revoked the free license for the Linux community. Suddenly, the thousands of developers building the Linux kernel had no way to collaborate.

Torvalds evaluated other open-source alternatives, but none met his strict criteria for the Linux kernel:

1. **Distributed workflow:** No central server dependency.
2. **Blistering performance:** It had to apply patches in seconds, not minutes.
3. **Cryptographic integrity:** It had to guarantee that code could not be corrupted or maliciously altered without detection.
4. **Strong defense against accidental corruption.**

Finding nothing suitable, Torvalds famously disappeared for a few days in April 2005. In a

frenzy of coding, he wrote the initial foundational scripts for a completely new system. He named it "**Git**" (British slang for an unpleasant or stubborn person, jokingly referring to himself). Within weeks, Git was functional enough to host the Linux kernel itself. Unlike previous tools that tracked files, Git was built fundamentally differently: it tracks snapshots of a massive directed acyclic graph (DAG) of file states, using SHA-1 cryptographic hashes to ensure absolute data integrity. What started as a desperate weekend hack to save Linux ended up becoming the undisputed standard for version control globally, wiping out nearly all proprietary competitors in the process.

A2. The license - What it actually says?

Git is licensed under the **GNU General Public License version 2.0 (GPL v2)**. To understand what this means, we have to examine the core tenets of free software which can be seen as follows.

The Four Freedoms of Free Software

The Free Software Foundation defines software as "free" if the end-user is granted four specific, irrevocable freedoms:

- **Freedom 0:** The freedom to run the program as you wish, for any purpose.
- **Freedom 1:** The freedom to study how the program works, and change it so it does your computing as you wish. (Access to the source code is a precondition for this).
- **Freedom 2:** The freedom to redistribute copies so you can help others.
- **Freedom 3:** The freedom to distribute copies of your modified versions to others. By doing this you can give the whole community a chance to benefit from your changes.

Git's GPL v2 license absolutely grants all four of these freedoms mentioned above. Anyone can download Git, read its C and shell source code, compile it, modify it to suit their own needs, and share those modifications in the community.

Can a company modify Git and sell it without sharing their changes?

The definitive answer is **No**. The GPL v2 is what is known as a "strong copyleft" license. It contains a viral clause: if you modify the GPL-licensed software and distribute that modified version (whether for free or for money), you *must* make the source code of your modifications available under the exact same GPL v2 license. A company cannot take Git, add a proprietary graphical user interface or performance upgrade to the core binary, and sell it as closed-source software. If they distribute the binary, they must distribute the modified source code. Thus ensuring that the software, and all its derivatives, remain permanently open.

MIT Licensing

The MIT License is a permissive software license originating at the Massachusetts Institute of Technology in the late 1980s. It stands as one of the most widely used permissive licenses in the open-source world. As a permissive license, it puts few restrictions on reuse and has high license compatibility. Unlike the GNU General Public License (GPL), it is a non-copyleft license, meaning it

imposes minimal restrictions on users. It grants complete freedom to use, copy, modify, and distribute the software, but crucially, it allows developers and corporations to take the code, improve upon it, and release their modified versions as closed-source or proprietary software without having to share those changes back with the community. This permissive nature makes the MIT License highly attractive for projects seeking maximum adoption and integration into commercial products, such as UI frameworks or small utility libraries, as it removes the fear of legal entanglement associated with strong copyleft clauses.

GPL vs. MIT Licensing: A Strategic Choice

The MIT License is fundamentally different. It is a "permissive" license. Here, permissive means Git provides an excellent example of "free as in freedom." It grants the same freedoms to use, modify, and distribute, but it *lacks* the copyleft clause. If Git were MIT-licensed, a corporation like Microsoft or Apple could take the code, improve it, and release a closed-source, proprietary version without ever giving their improvements back to the community.

If I were building a completely new software project today, my choice of license would depend entirely on my strategic goals.

- If my goal was **maximum widespread adoption and integration**—for example, a JavaScript UI framework or a small utility library—I would choose the **MIT License**. I would want corporations to use it without fear of legal entanglement, knowing that widespread use makes my tool the industry standard.
- However, if I were building a **foundational infrastructure tool** like an operating system, a database, or a version control system like Git, I would choose the **GPL**. This forces a reciprocal relationship: corporations can profit from my work, but they must contribute their bug fixes and feature additions back to the communal pool.

"Free as in Free Beer" vs "Free as in Freedom"

This distinction is crucial in open source.

- The term "free beer" (Gratis) refers specifically to cost. For instance, a proprietary mobile game qualifies as "free beer" because it can be downloaded at no expense; however, users are prohibited from viewing, altering, or studying its underlying source code.
- "Free speech" (Libre) means liberty. It represents the fundamental right of users who can control their own computing rather than being controlled by the software developer. In the context of Git, this liberty ensures that the global developer community is never beholden to a single entity for their version control needs. Unlike "free beer," which only addresses the lack of a price tag, "free speech" guarantees that the software remains a public good that can be audited, improved, and shared by anyone.

Git provides an excellent example of "free as in freedom." Yes, Git costs $0 (free beer). But more importantly, when GitHub (a multi-billion dollar platform) needed Git to scale to handle massive repositories, they didn't have to wait for Linus Torvalds to build those features. They exercised their *freedom* to modify the Git source code, optimize the packing algorithms, and submit those patches back to the Git mailing list. That is freedom.

A3. The ethics of open source

The philosophy of open source extends far beyond legal documents. It asks deep reasoning and ethical questions about how human knowledge and their should be treated and managed.

Should all software be open source?
Well this is a complex ethical debate which doesn't has a fix answer. Its answer varies according the situations and usage patterns of how the software will be used.

- **The argument for YES:** Proponents argue that software code is essentially a form of shared mathematical knowledge; therefore, restricting access to it is seen as inherently unethical and a hindrance to human progress. In an increasingly algorithmic world, closed-source software often functions as a "black box," forcing users to place blind trust in corporations regarding their data privacy and system security without any means of independent verification. Furthermore, proprietary models lead to massive economic and intellectual inefficiency through duplicated effort, as thousands of separate companies spend vast resources rewriting the same foundational boilerplate code instead of collaborating on a single, robust, and transparent solution for the common good.
- **The argument for NO:** The primary counterargument centers on economic incentives and liability. Developing highly specialized software—such as proprietary algorithms for pharmaceutical drug discovery or advanced aerospace flight control systems—requires massive capital investment and years of research. Without the temporary monopoly granted by proprietary software and copyright protections, companies might not invest billions in R&D if competitors could immediately copy their work for free. Furthermore, in highly regulated industries like medical devices or banking, a single corporate entity must maintain strict, closed control over the codebase to ensure absolute liability, regulatory compliance, and accountability for critical failures that could result in significant legal or physical harm.

Although the debate about whether software should be open source or closed can be explored extensively, it is often a matter of context rather than a binary choice. Both models serve distinct needs depending on the situation. Open source thrives in collaborative environments where transparency and shared innovation are paramount, while closed source provides a framework for protecting massive R&D investments and ensuring strict liability in highly regulated industries like medical devices or banking. Ultimately, both methodologies are essential to the modern digital landscape, addressing different requirements for security, economic incentive, and public accessibility.
.

Is it ethical for a company to profit from free community labor?
Consider companies like Red Hat, Google, or GitHub (Microsoft). GitHub generates billions in revenue entirely by wrapping a proprietary web interface around Git—a tool built by unpaid community volunteers. Is this ethical?
From a strictly legal standpoint, yes; the GPL permits commercial use. From an ethical standpoint, it is nuanced. It becomes unethical—or at least highly parasitic—when corporations strip-mine open-source projects without giving back. However, the ecosystem usually finds a balance. Microsoft and Google now

employ thousands of developers whose sole full-time job is to contribute upstream to open-source projects like Git and Linux. They realize that if the foundation crumbles, their profitable products collapse. The relationship is symbiotic: the community provides the raw innovation, and corporations provide the capital to scale, secure, and maintain it.

Standing on the shoulders of giants

Isaac Newton famously said, "If I have seen further, it is by standing on the shoulders of giants." In software, open source is the embodiment of this quote. Linus Torvalds did not have to write the cryptographic hashing algorithms used in Git; he utilized standard libraries like SHA-1. He didn't have to write the network transport layer; he used SSH and cURL.

Open source categorically enables original innovation because it eliminates the "friction of the starting line." A computer science student in a dorm room today can pull down the exact same machine-learning frameworks (TensorFlow, PyTorch) used by Google's top engineers. Because they don't have to spend five years reinventing the foundational math, they can immediately start innovating on the *application* of that math. Open source ensures that humanity only has to solve a foundational computer science problem once.

Part B: Linux Footprint (Unit 2)

To understand Git practically, I audited its presence on a standard Ubuntu Linux environment (24.04 LTS).

Installation Methods Across Platforms

Git is a cross-platform tool available for all major operating systems. Below are the standard methods to install and maintain Git on Windows, macOS, and Linux.

- **Windows**: On Windows, the most common method is using the Git for Windows installer (BASH, GUI, and Shell integration). Users can download the executable from the official website and follow the wizard. Alternatively, Windows users can use the winget package manager: ```winget install --id Git.Git -e --source winget.```
- **MacOS**: MacOS users can install Git by simply running `git --version` in the terminal; if not present, it will prompt to install the Xcode Command Line Tools. For a more up-to-date version, the Homebrew package manager is recommended: `brew install git`.
- **Linux (Ubuntu/Debian)**:Git is deeply integrated into almost all Linux distributions. On Debian/Ubuntu systems, the primary installation method is via the package manager:

  ```sudo apt update && sudo apt install git```

  For users requiring the absolute latest features or custom configurations, Git can be compiled directly from source. This involves downloading the tarball from kernel.org and running:

  ```make configure && ./configure --prefix=/usr && make all && sudo make install```

Key Directories (Linux)

Once installed, Git distributes its files across the Linux filesystem hierarchy in accordance with standard

POSIX guidelines:

- **Binaries:** The main executable is located at /usr/bin/git. However, Git is actually a suite of dozens of smaller sub-programs (like git-commit, git-merge). These helper binaries are traditionally stored in /usr/libexec/git-core/.
- **Configuration:** System-wide configuration (affecting all users) is stored in /etc/gitconfig. User-specific configuration (name, email, aliases) is stored in ~/.gitconfig or ~/.config/git/config. Repository-specific configuration is stored locally inside the project folder at .git/config.
- **Data Storage:** Git does not use a centralized database like MySQL. All version history, objects, and logs for a project are stored locally within the .git hidden directory at the root of the user's project folder.

User and Group

Unlike web servers (Apache) or databases (MySQL), Git does not run as a continuous background daemon under a dedicated service account (like www-data). Git executes strictly as the **invoking user**. If I run git commit as the user student, the resulting files created in the .git/objects directory will be owned by student:student.

This matters immensely for security as Git runs with the permissions of the user executing it, it cannot accidentally overwrite system-critical files in /etc unless the user recklessly runs Git as root.

Service Management

Because Git is a user-invoked command-line utility, it is not managed via systemd (e.g., you cannot run systemctl start git). The exception to this is git daemon, a simple server that allows unauthenticated fetching of repositories, but this is rarely used today in favor of accessing Git securely over standard SSH daemons (sshd).

Update Model

Security patches and updates for Git are handled upstream by the Git core maintainers. They release a new version, which is then fetched, tested, packaged, and signed by Linux distribution maintainers (like Canonical for Ubuntu or Red Hat for Fedora). The user receives these updates simply by running standard system updates (apt upgrade). This ensures cryptographic trust via the package manager's GPG keys.

Part C: The FOSS Ecosystem (Units 3 & 4)

No open-source project exists in a vacuum. Git is deeply interwoven into the broader Free and Open Source Software (FOSS) ecosystem.

Dependencies

Git is written primarily in C and shell scripts, designed to be brutally fast and lightweight. However, it relies heavily on other foundational FOSS libraries:

- **zlib:** For compressing the objects (commits, trees, blobs) to save massive amounts of disk space.
- **libcurl:** For handling HTTP/HTTPS network transfers when pushing or pulling from remote servers.

- **OpenSSL / GnuTLS:** For establishing secure encrypted connections.
- **OpenSSH:** While not technically a library dependency, Git relies almost entirely on the SSH protocol to securely authenticate users and transfer data without needing its own proprietary authentication daemon.

What has Git enabled or inspired?
Git's impact is incalculable. It directly enabled the creation of **GitHub** (launched in 2008), which fundamentally changed software from a solitary engineering task into a social network. It spawned competitors and alternatives like **GitLab** and **Bitbucket**.
More importantly, Git inspired an entire movement known as **GitOps** and modern CI/CD (Continuous Integration / Continuous Deployment). Because Git tracks state perfectly, tools like Jenkins, GitHub Actions, and ArgoCD now use Git repositories as the "single source of truth" to automatically build, test, and deploy software to the cloud whenever code is merged.

Where does it fit in the LAMP stack?
Git is not formally part of the LAMP stack (Linux, Apache, MySQL, PHP/Python). However, it is the invisible layer that manages the stack. A modern developer does not FTP files directly to an Apache server anymore. Instead, they write their PHP code locally, commit it to Git, and push it to a repository. A deployment script then pulls the latest Git commit directly onto the LAMP server. Git acts as the transport and versioning layer for web development.

Community and Governance
The Git community is highly decentralized and extremely technical. It does not use GitHub for its own development; rather, true to its roots, the Git project itself is governed via a mailing list (git@vger.kernel.org). Developers submit patches as plain text emails.
The project is not controlled by a corporation. Following Linus Torvalds' departure from active Git maintenance, the project was handed over to **Junio C Hamano**, who has been the core maintainer since 2005. The governance is a meritocracy: patches are fiercely debated, heavily scrutinized, and only merged if they prove to be mathematically and architecturally sound.

Part D: Open Source vs Proprietary (Critical Analysis)

To understand Git's true value, it must be compared against a proprietary alternative. In enterprise environments, the closest proprietary alternative to Git is **Perforce (Helix Core)**.

Dimension	Open Source (Git)	Proprietary Alternative (Perforce Helix Core)
Cost	**Free.** $0 licensing fees, regardless of the number of users or repository size.	**Expensive.** Charges high per-user, per-year licensing fees after a small free tier.

Dimension	Open Source (Git)	Proprietary Alternative (Perforce Helix Core)
Security (Code Auditing)	**High.** Millions of developers can inspect the C code for vulnerabilities. Hashes guarantee integrity.	**Dependent on vendor.** Users cannot audit the source code. Must trust Perforce's internal security team.
Support and Reliability	**Community-driven.** Support comes from StackOverflow, forums, and documentation. Very high reliability.	**Corporate SLAs.** Guaranteed response times, dedicated account managers, and formal technical support.
Freedom to Modify	**Absolute.** Anyone can fork Git, change its algorithms, or build custom front-ends without asking permission.	**Zero.** The software is a closed binary. Modifications are legally prohibited and technically impossible.
Community vs Corporate	Governed by an open mailing list. Features are added based on global community consensus and need.	Features are dictated by a product roadmap managed by executives, often prioritizing high-paying enterprise clients.

Verdict

When comparing Git to proprietary solutions like Perforce, the dominance of open source becomes overwhelmingly clear for standard software development. Git's decentralized nature fundamentally aligns with how modern developers work: offline capable, branch-heavy, and experimental. The zero-cost barrier to entry allowed it to become the universal language of version control, meaning almost every graduating computer science student already knows how to use it, drastically reducing onboarding time for companies.

However, it is important to acknowledge that Git is not perfect for *every* situation. Proprietary tools like Perforce still maintain a strong grip on the video game industry because Git struggles natively with massive, gigabyte-sized binary files (like 3D models and textures), whereas Perforce was designed to stream large binaries centrally. Yet, even here, the open-source community responded by building Git LFS (Large File Storage) to bridge the gap. Given everything I have researched, I would unequivocally recommend Git for real-world deployment in 99% of software scenarios. It is robust, mathematically sound, and represents the absolute best of what open-source collaboration can achieve. I look forward to using it daily and contributing back to the tools that surround it.